

DEEP DIVE

Everyone Hits the Wall

Adoption of coding agents follows the same five stages almost everywhere: a few enthusiasts, a burst of output, a stall, a rebuild, and something steadier and narrower than the burst promised. Most teams are at the stall. It is where the value of the whole exercise gets decided.

21 August 2026 | Issue 23 | Engineering Digest | Deep Dive

15 - 21 AUGUST 2026

5

STAGES OF THE
ARC

16

TEAMS &
PRACTITIONERS

19

STUDIES &
DATASETS

1

NAMED BANK
ROLLOUT

90%

Nine in ten professional developers now use a coding agent at work at least weekly. That settles the adoption question and opens every other one. JetBrains, August 2026.

FOREWORD

Nine in ten professional developers now use a coding agent at least weekly. Adoption is finished as a question. It explains almost nothing about why two teams holding the same tools are having completely different years — what a team is experiencing depends on how far along a fairly predictable arc it has travelled.

This issue maps that arc in five stages: experimenting, the honeymoon, the wall, the rebuild, and settled practice. Most teams are in the third stage. The evidence for each comes from

named engineers and organisations who published what they actually did, set against a year of studies measuring what it produced.

HOW TO READ THIS

Three Paths Through This Issue

2 min

THE SHAPE

The cover figure, the adoption-arc diagram in Section 1, and the verdict box in Lessons & Implications.

12 min

WHERE MOST TEAMS
ARE

Add Stage Three, the wall, and Stage Four, the rebuild — what stalls and what the teams that got past it changed.

27 min

THE DEEP READ

All five stages, the Monzo case study, Lessons & Implications, then Meanwhile, On the Radar and Recommended Reading.

1 OF 8

THE SHAPE

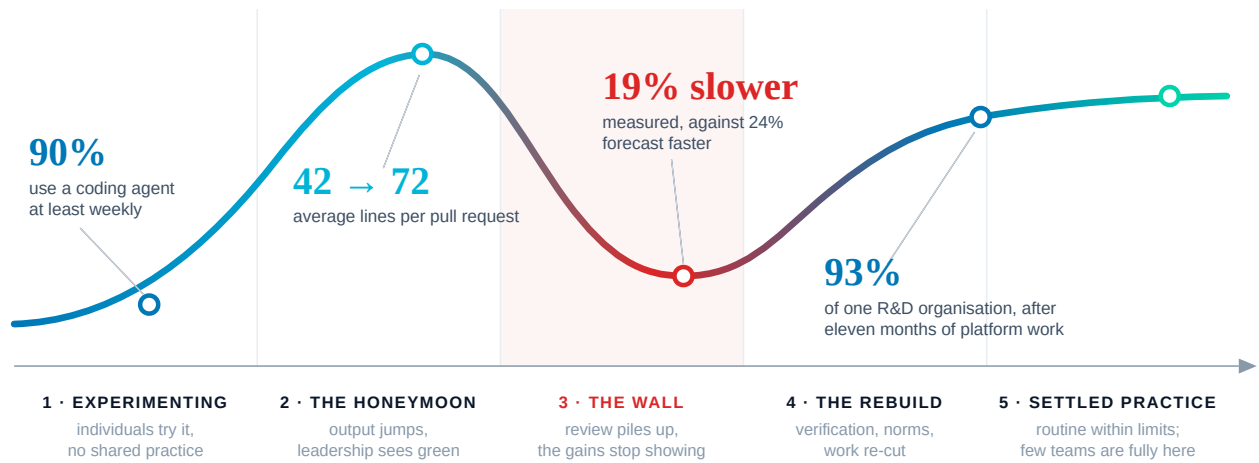
Rollouts are converging on the same five stages

(2 min)

Enough organisations have now published what they did with coding agents that their rollouts can be laid side by side. They do not look alike in their tooling. They look remarkably alike in their sequence. A team starts with a few individuals trying things on their own, gets a burst of output that feels transformative, then spends a long stretch unable to show the gain anywhere it can point to. The teams that come out of that stretch have rebuilt their verification and their working norms, and what they arrive at is steadier and considerably narrower than the burst promised.

The adoption arc

Five stages teams pass through, with the year's evidence pinned to the stage it belongs to



Horizontal axis is months into a rollout. The curve is schematic; the four pinned figures are measured. Sources: JetBrains, August 2026 · DX, Q2 2026 · METR, July 2025 · Cloudflare, April 2026.

Nothing is awarded for reaching the end of this arc. The fifth stage describes what a small number of organisations report today, not a destination with a date on it. What the arc is good for is location: a team that knows which stage it is in spends much less time arguing about which tool to buy.

READING THE NUMBERS

“Agent” means three different things across the sources here. An **assistant** completes the next line while a person types. **Agent-mode** means the agent takes a whole task and runs it, while a human still starts it, reviews it and merges it. An **autonomous agent** acts without per-step approval. Almost everything in this issue is agent-mode. Where a figure counts something else — an assistant, or an organisation rather than a person — the text says so at that point, and nowhere else.

Most teams are in the third stage. It gets the longest treatment here for that reason, and because everything a team builds afterwards is a response to something it found there.

2 OF 8

STAGE ONE

Experimenting: a few people, a lot of scattered wins (2 min)

Nine in ten professional developers now reach for a coding agent in a given week [JetBrains, August 2026]. The figure counts people, not teams. An engineer who tried one on Tuesday and told nobody is inside it. So is an engineer on a team that has rebuilt its entire review process around agents. Counted this way, adoption says very little about whether an organisation has a practice.

The gap shows up as soon as anyone looks at the work rather than the headcount. In an analysis of agent-written pull requests published this summer, seven projects in ten had fewer

than one contributor in five taking part in an agentic workflow at all [LeadDev, July 2026]. Most people on most of those projects were outside the practice while their organisations counted themselves adopted.

From the inside, stage one looks like two or three engineers producing results nobody else can reproduce, no agreement on which tool, and no written record of what worked. The wins are real and they are personal. They do not transfer, because nothing has been written down for them to travel through.

The second-order problem is divergence. Everyone develops their own habits for prompting the agent and correcting it, so a team that started with one shared process quietly ends up with five. It stays invisible until a handover or an incident, at which point it turns out that two engineers on the same service hold completely different beliefs about what the agent is allowed to touch.

The engineers who are not interested are usually not persuaded by being asked to let a machine write their code. Mitchell Hashimoto, who co-founded HashiCorp and now builds the Ghostty terminal, has the most portable move for that conversation. “There’s a lot of people like, ‘I don’t want it to write code for me.’ But just delegate some of the research part.” Comparing libraries and enumerating edge cases sidesteps the identity objection entirely, and it is work that needs no real-time attention.

The stage ends when one person’s result becomes visible enough that the rest of the team copies it. That is almost always a demo, and it is almost never planned.

Sources: [JetBrains developer research](#) (August 2026) · [Chris Stokel-Walker, LeadDev](#) (28 July 2026) · [Gergely Orosz on Mitchell Hashimoto, The Pragmatic Engineer](#) (25 February 2026)

3 OF 8

STAGE TWO

The honeymoon: everything gets faster, and the graphs agree (2 min)

One line from Craft Docs is the sort of result that ends an argument inside a company: “This migration work was estimated to take 3-4 months. It was done in a week with AI tooling.” Stage two is what happens in the months after a result like that lands. The scepticism drains out of the room, the tools get bought for everybody, and for a while the work genuinely does feel different.

Parallel agent use arrives on its own, and the mechanism is dead time. Ty Smith, a principal engineer at Uber, described it without dressing it up: “as an engineer, you’re giving a prompt and waiting for something. While it’s running and you’re waiting you’re thinking: ‘What am I

going to do? Have a coffee or browse Reddit? Might as well kick off another background agent.” Three agents at once is what happens when waiting is boring.

The volume shows up quickly. Across the engineering organisations DX measures, the average pull request has grown from 42 to 72 lines, and more than half of all merged code is now AI-authored [DX, Q2 2026]. Bigger changes, more of them, and the majority of new material written by something that will not be sitting in the review.

Throughput measures agree. Microsoft researchers who studied their own employer’s command-line agent rollout found that adopters “merged roughly 24% more pull requests than they would have otherwise”. They attached the caveat themselves, which is the model for how any throughput number should travel: “We use merged pull requests as our proxy for output -- acknowledging that a merged PR is not the same as the value it delivers -- and the lift persists across our four-month window.”

That distinction is the one nobody is looking at during stage two. A merged pull request measures work leaving the team. Value arriving at a customer is a different quantity, and the two only track each other while somebody downstream still has capacity to check the difference. This is the stage at which AI budget gets approved without much argument, because the anecdotes are strong and every dashboard leadership already had points the right way.

Nothing about stage two is fake. The output really is larger, it really does arrive faster, and the engineers producing it are not imagining the change. The stage ends when that output reaches the people who have to check it.

Sources: [Gergely Orosz on Craft Docs, The Pragmatic Engineer](#) (27 January 2026) · [Gergely Orosz on Uber, The Pragmatic Engineer](#) (10 March 2026) · [DX Q2 2026 benchmarks](#) (14 August 2026) · [Murphy-Hill, Butler & Savelieva, arXiv preprint](#) (1 July 2026)

4 OF 8

STAGE THREE

The wall: the output arrives, and the gains stop showing (4 min)

The wall arrives as a review queue. Addy Osmani, an engineering leader at Google, relays one developer’s line about reviewing an agent’s pull request — that it made them “the first human being to ever lay eyes on this code” — and then draws out what follows from it.

“In normal review the author already understood the change and you were checking their work. With an agent PR, nobody has reconstructed the why yet,

and the reviewer is the first to try.”

— Addy Osmani, *Agentic Code Review*, 15 June 2026

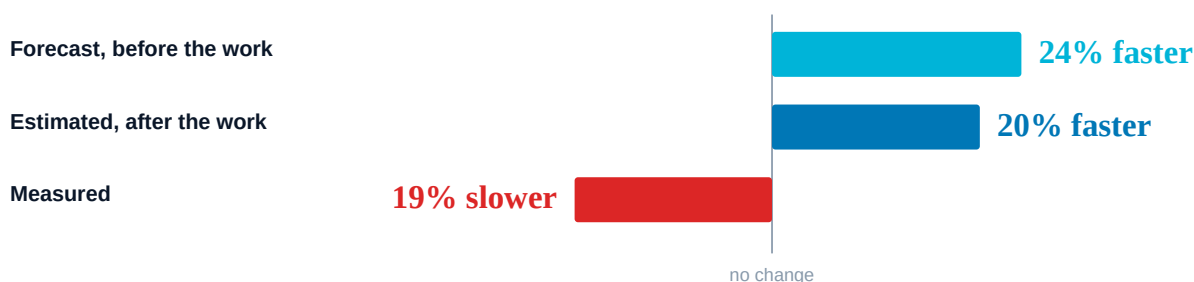
That is the whole mechanism. Reviewing a colleague's change means checking work somebody has already understood. Reviewing an agent's change means building that understanding from nothing, in the reviewer's own head, for the first time. It is a slower and more expensive job, which is why review minutes stop dividing the way any capacity plan assumed they would. Larger diffs make it worse in a specific way: an oversized change does not get reviewed more carefully, it gets rejected outright or waved through.

The norm most teams believe they hold is the first thing to break. In 79% of agent-written pull requests studied this summer, the same developer both reviewed and modified the agent's contribution, and just one workflow in eight involved more than one human [LeadDev, July 2026]. Nobody voted to remove the second pair of eyes. It came off under load.

The felt speed does not survive measurement either.

Faster, expected. Slower, measured.

Change in how long the same tasks took, experienced developers on repositories they knew well



Source: METR randomised controlled trial, 16 experienced open-source developers, 246 tasks, published 10 July 2025. Tooling of that generation; the gap between sen:

The sixteen experienced open-source developers in METR's randomised trial, working across 246 tasks in repositories they knew well, expected agents to make them about a quarter faster. Afterwards they believed they had been about a fifth faster. They were a fifth slower. The study is a year old and the models have moved on; what has not moved is that the sensation of speed is a poor instrument for measuring it.

That gap is why the wall arrives as a surprise. A team that trusts the sensation plans its next quarter on it, and then finds the quarter did not land where the feeling said it would.

Quality measures are flat while volume climbs. Chris Wysopal of Veracode, which sells application security, put the year's security finding in a single line: “[M]odels may be almost syntactically perfect, but they’re still failing on nearly half of all tasks where security is needed.” The security pass rate for generated code sits at 56% and has not improved while models have got better at nearly everything else. For a team in stage three, that means every extra pull request arrives at a queue that now has to carry a security judgement it did not carry before.

Maintainability moves the same way. GitClear, which sells code analytics, compared heavy AI users with engineers who barely use the tools, across four years of commit history: the heavy users produce four to ten times as much code, for about 25% more velocity — and most of that

output gap already existed before AI arrived, so the tools widened an old pattern rather than created it. Its own summary is the sentence a team at the wall recognises immediately.

“The throughput is real, but so is the debt it accrues” — GitClear, *The Maintainability Gap*, January 2026.

Armin Ronacher, the creator of Flask and formerly a principal architect at Sentry, describes what the extra code is actually made of. “Present-day models tend to produce code that is too defensive, too complex, too local in its reasoning. They avoid strong invariants. They add fallbacks instead of making bad states impossible. They duplicate code, invent bad abstractions, and paper over unclear design with more machinery.” Each pass adds one more small defence. The system looks progressively more robust and becomes progressively harder to understand.

The bill arrives in the same window. Vitaly Gordon, chief executive of Faros AI, relayed a chief technology officer’s problem: “One of my engineers spent \$40,000 on tokens last month, and I genuinely don’t know whether I should stop him or should I go and tell everyone else to be like him.” What makes that number unmanageable is attribution. Nobody can tell whether the engineer is the organisation’s best investment or its worst, because nothing was built to say so. J.R. Stormont of the FinOps Foundation dates the turn precisely: “We started hearing existential crises, and the whole conversation shifted from tokenmaxxing and ‘go fast’ to ‘we need guardrails, how do we control this?’”

Then adoption stops climbing. Sentry unlocked budget, relaxed its intellectual-property restrictions and opened frontier tools to every engineer, and its co-founder and chief product officer David Cramer published what happened next: “We’ve also been very intentional about adoption, but so far we haven’t seen the organic growth we’d like in the company. About half of engineering is using AI tools in their daily workflows, but only a small subset of engineers have gotten truly comfortable with the state of the art.” Half the engineers use the tools daily. The other half still do not, and that split has held for months. Removing every barrier a company can name does not produce a practice.

The experience measure moved with all of it. Across DX’s benchmarks the average Developer Experience Index — its composite score for how engineering work feels to the people doing it — slipped from 67 to 65 [DX, Q2 2026]. Two points is a small move. It is a small move in the wrong direction, in the year the tooling improved most.

Stage three is where most teams are. It is also where the value of the whole exercise gets decided, because everything worth building afterwards is a response to something found here.

Sources: [Addy Osmani](#) (15 June 2026) · [Chris Stokel-Walker, LeadDev](#) (28 July 2026) · [METR](#) (10 July 2025) · [Veracode](#) (28 July 2026) · [GitClear](#) (January 2026) · [Armin Ronacher](#) (23 June 2026) · [Rebecca Bellan, TechCrunch](#) (5 June 2026) · [David Cramer, Sentry](#) (16 January 2026) · [DX Q2 report](#) (2026)

STAGE FOUR

The rebuild: what the teams that got through actually changed (6 min)

The organisations that published an account of getting past the wall did four things, in roughly this order. They made verification the gate. They wrote down the practice they had been improvising. They re-cut the work to suit the way agents actually run. And they fixed the mechanics of the rollout itself. None of it is exotic, and Cloudflare's engineers said so about their own stack: "None of these pieces are especially novel on their own. Plenty of companies run service catalogs, ship reviewer bots, or publish engineering standards. The difference is the wiring."

Verification becomes the gate

The historical objection to writing tests was the engineer time they cost to write and maintain. Simon Willison, an independent developer best known for Django and Datasette, says that objection has gone: "They're free now. They're effectively free. I think tests are no longer even remotely optional." The reason they matter has changed too. A test suite is now the mechanism by which an agent can tell whether it has finished. A team that skipped tests has no such mechanism, and no amount of instruction-writing substitutes for it.

Willison opens every agent session the same way — point the agent at the command that runs the tests, then name the discipline, red-green, meaning write the failing test first and then make it pass. Every capable agent already knows the term, so the convention costs a handful of words and is a team convention rather than a prompt library. He is equally clear about its limit: a passing suite does not prove the service boots, so the agent is made to exercise the running system and leave a written record a reviewer can read.

Figma's security engineers state their merge gate in one sentence: nothing merges without human review, a static-analysis pass and an agentic review, with failover between model providers so that an outage at one vendor cannot let a pull request through unreviewed. Two frontier models run independently, and either one raising a finding escalates it, on the deliberate grounds that they miss different bugs. The whole apparatus is checked against a fixed corpus of known cases — the practice teams now call evals — and it is affordable: "a pull request review runs about \$0.50 median spend, rarely more, since most PRs have nothing to flag."

Writing the policy that drives all of it produced something the team had not planned. "Ninety-nine lines, 2,560 words, and 68 precedents later, this work had a side effect we did not plan for: We had written a complete threat model, in roughly the form we'd want a new hire to read on

day one. The policy is the threat model.” The artefact that makes agents useful turned out to be the artefact the organisation had needed for its humans all along.

Birgitta Böckeler of Thoughtworks gives the whole layer a name teams can budget for. A *harness* is everything in an agent that is not the model: instruction files, bootstrap scripts, linters, type checkers, test suites, review agents. Guides steer the agent before it acts; sensors tell it whether what it did was right. Most teams over-invest in guides and under-invest in sensors, and a sensor that only fires in CI has fired too late to correct anything.

“A good harness should not necessarily aim to fully eliminate human input, but to direct it to where our input is most important.”

— Birgitta Böckeler, Thoughtworks, *Harness engineering for coding agent users*, 2 April 2026

The practice gets written down

Cloudflare traced its agents’ bad output to a specific cause and fixed it with a file. “Usually the problem was local context: the model didn’t know the right test command, the team’s current conventions, or which parts of the codebase were off-limits. That pushed us toward AGENTS.md: a short, structured file in each repo that tells coding agents how the codebase actually works and forces teams to make that context explicit.” AGENTS.md is a convention: a plain file in the repository root that agents read before they start. The forcing function is the point: a team cannot write down which parts of the codebase are off-limits without first agreeing on the answer.

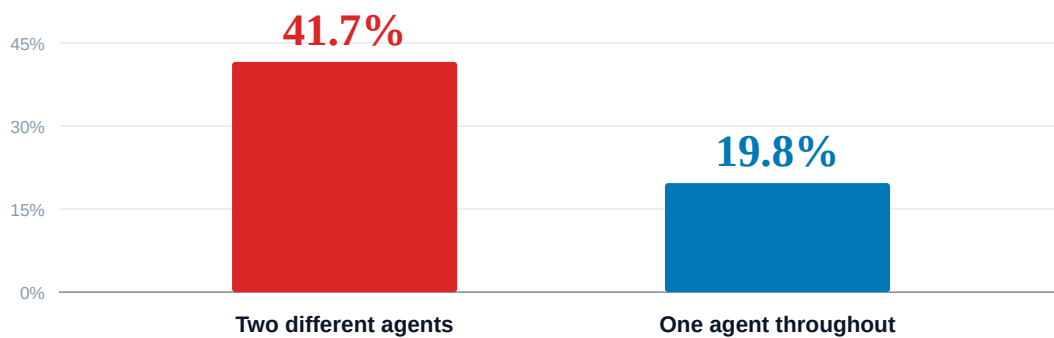
Teams that got through the wall also wrote down where they do not use agents. Ronacher draws the line on the shelf life and invariant density of the output rather than on the size of the task. Unsupervised loops work well on transformations of code that already exists, such as ports and migrations, and on deliberately short-lived artefacts such as performance experiments and security sweeps. They work badly on new, long-lived code carrying load-bearing invariants, and he says so about his own work: “My current status is that I have not had much success with this way of working for code I deeply care about which turns out to be quite a lot of code.”

Hashimoto’s rule for the working day is the most transplantable norm in the record, and it fits on a calendar: “If I’m coding, I want an agent planning. If they’re coding, I want to be reviewing.” The human and the agent are never in the same mode at the same time. Slow work gets queued at transitions — before a meeting, at the end of the day — because research and edge-case enumeration need no real-time attention.

One more norm is worth setting before it is needed. Where two different agents work the same repository at the same time, their changes collide far more often than two runs of the same agent do.

When two agents work the same repository

Share of overlapping pull-request pairs that conflict when merged



Source: Xu, Subramanian & Karthik, arXiv, 6 July 2026. Cross-agent overlap is rare today: 0.5% of concurrent pairs, in about 4.3% of repositories.

The honest reading is forward-looking. Two different agents rarely meet in the same repository today, so this describes what happens when they do rather than what happens in a median week. It is the cheapest argument available for settling on one agent per codebase before multi-agent overlap becomes normal.

The work gets re-cut

Planning is the ceremony that breaks first. Fiona Fung, director of engineering for Claude Code at Anthropic, is direct about it: “When I first joined the Claude Code team, we wrote a pretty good six month roadmap, and then because of Claude Code, so many things changed that it was out of date by month three.” What replaced it has a name and a shape. “I call it just-in-time (JIT) planning, almost like JIT compiling: how do you do just the right amount at the right time? Our planning ritual shifted away from design docs toward discussions in PRs or prototypes.” Rather than reviewing a proposal, the team builds it, puts internal users on it and acts on what comes back.

Review gets split by category rather than by seniority. In Fung’s account the machine handles style, linting, pull-request feedback, catching bugs before a commit and adding tests, and “Where we still definitely want a human is expertise” — legal risk tolerance, trust and safety, product calls. That split is what makes a review queue survivable, because it stops human attention being spent on the things a machine reliably catches.

Which work goes first is a decision with a defensible answer. Uber sent the toil, and Anshu Chada, an engineering director on its developer platform, described what came back: “What we found is when we push some of the boring stuff to AI – upgrades, migrations, bug fixes – not only does it result in much higher satisfaction from our engineers, but they’re able to push our product and create features for end users in ways that we didn’t even think were possible.” Morale is the point as much as throughput. Satisfaction early is what buys the credibility to attempt harder work later.

The rollout mechanics get fixed

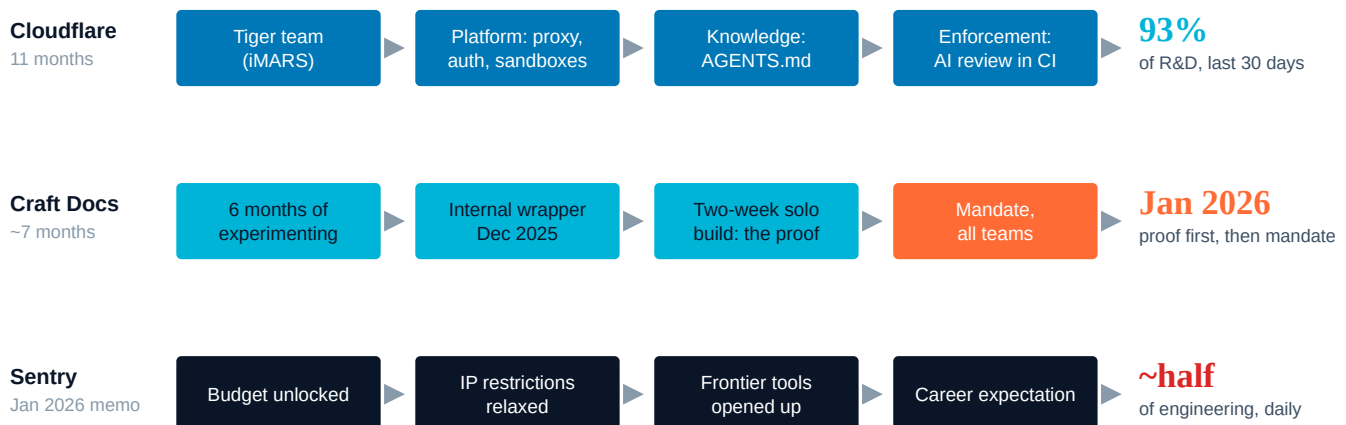
Cloudflare names one early decision as the one it got right, and it is an unglamorous one. Every AI request routed through a single proxy service from the first day, when connecting clients straight to the model gateway would have been quicker to stand up. “But centralizing through a Worker meant we could add per-user attribution, model catalog management, and permission enforcement later without touching any client configs. Every feature described in

the bootstrap section below exists because we had that single choke point.” Every organisation that skipped the choke point rediscovered the need for it when the token bill arrived.

Shopify reached the same architecture from the opposite direction. Its default is one tool for one job, and AI is the stated exception: “At Shopify, we always have one tool for one job—except for with AI,” explains Farhan Thawar, its head of engineering, “since we don’t know yet which company, workflow, or model is going to win.” Engineers run several agents side by side. What makes plural tooling survivable is a singular layer beneath it: one proxy carrying cost control, per-team usage analytics and the ability to switch models without touching anybody’s setup.

Three rollouts, in the order they took their decisions

Same destination, three sequences, three different positions reached



Sources: Cloudflare engineering blog, 20 April 2026 · The Pragmatic Engineer on Craft Docs, 27 January 2026 · David Cramer, Sentry, 16 January 2026.

Cloudflare built the platform layer before the context layer and the enforcement layer. Craft Docs earned an internal proof before issuing a mandate. Sentry removed every access barrier it could name and reports a plateau anyway, which is why a rollout has to be run as a practice problem as well as an access one.

Craft Docs ran the sequence in the order most companies invert. Leadership refused to commit until it had its own demonstration, so six months of experimentation and a two-week internal build came before mandatory adoption across every team in January 2026. The ritual it kept is the most stealable artefact in the account: a weekly challenge in which teams propose something they believe would take two to three months, then attempt it in a week. The demonstration belongs to the team, which is why it converts sceptics.

Ramp’s contribution is a clock. Major features ship daily to an early-access tier and advance to general availability on evidence — demos, key metrics, customer feedback and a rollout plan — with the reviewing group committed to responding within 48 hours or letting the feature ship regardless. A gate with a deadline attached is a gate that speeds work up.

What actually spreads a rollout is the finding most enablement budgets get wrong. Microsoft’s researchers found that first use spread primarily through social networks, while whether people stayed tracked how much they actually coded rather than who they were. Trying and sticking are two different problems. Visible peer use gets an engineer to try; whether the tool fits the shape of their work decides whether they keep going. Uber, which has the leverage to mandate

if it wanted to, reached the same conclusion from practice and found adoption slower than expected even so.

Sources: [Cloudflare](#) (20 April 2026) · [Simon Willison](#) (14 March 2026) · [Figma](#) (23 July 2026) · [Birgitta Böckeler, martinfowler.com](#) (2 April 2026) · [Armin Ronacher](#) (23 June 2026) · [The Pragmatic Engineer on Mitchell Hashimoto](#) (25 February 2026) · [Xu, Subramanian & Karthik, arXiv](#) (6 July 2026) · [Fiona Fung, Anthropic](#) (3 June 2026) · [The Pragmatic Engineer on Uber](#) (10 March 2026) · [Bessemer Atlas on Shopify](#) (2 April 2026) · [The Pragmatic Engineer on Craft Docs](#) (27 January 2026) · [Bessemer Atlas on Ramp](#) (10 June 2026) · [Murphy-Hill, Butler & Savelieva, arXiv preprint](#) (1 July 2026)

6 OF 8

STAGE FIVE

Settled practice: narrower than the honeymoon promised (3 min)

Very few organisations are here, and the honest description of the stage is a modest one. Agents are routine for defined classes of work — migrations, upgrades, test authorship, first drafts, research — and unremarkable within them. Nobody demos them any more.

Cloudflare reports that 93% of its research and development organisation used AI coding tools in the last thirty days, running on infrastructure it built itself [Cloudflare, April 2026]. That figure took eleven months of platform work by a temporary tiger team, after which ownership landed permanently with the developer-productivity team that already ran CI/CD and build systems. The number is what the platform work produced.

Verification stays the constraint on the far side. Nothing in the rebuild removes the review gate; its purpose is to spend human attention only on the judgements humans have to make. Every organisation here that runs agents at volume still has a person merging the result.

Autonomy remains rare and deliberate. Around 11% of organisations report agentic AI running in production rather than in a pilot [Deloitte, 2026], while 28% of developers describe autonomous AI as their primary way of working [GitKraken, 2026] — a report from a vendor that sells the review tooling it identifies as the new bottleneck. The two figures count different things: one counts organisations describing their systems, the other counts individuals describing their own habits. Between weekly use at ninety per cent and production autonomy at eleven sits a very large supervised middle, and that middle is what settled practice actually looks like.

The open worry is what happens to the people who have not yet learned to judge the output. Sarah Wells, a software engineering consultant formerly of the BBC and the Financial Times, names it plainly about junior engineers. “I’m worried that they’re not; they can’t make the judgement about whether the agent is making a sensible suggestion,” she says. Juniors are

substituting the agent for the conversation with a colleague, and they are the least equipped to tell whether the answer they got was any good.

The channel that used to fix that is closing. “I think code reviews are probably going to stop being the way that you share knowledge,” Wells says. “You’re going to share knowledge at a higher level than the code itself.” Collaboration that used to happen by accident inside review has to be put back deliberately somewhere else — design, architecture, incident review — or it does not happen at all.

Senior engineers carry a version of the same problem. Farhan Thawar names comprehension debt as the risk that worries him most: as agents write more, engineers gradually lose their understanding of the systems they are responsible for. “The brain is a muscle,” he says, “If you stop going to the gym—or stop using your brain—it will atrophy.” Ronacher’s personal bar is the statable version of the countermeasure: being able to explain what the system does under pressure without asking the model first.

The regulated version of stage five is still in research. Lloyds Banking Group is running a four-year partnership with the University of Glasgow that places agents directly inside software and data engineering teams and measures them alongside human engineers under live conditions. QA Financial, reporting the programme in May, described the shift in evaluation it represents: “Lloyds’ move shifts the testing question from ‘does this work in isolation?’ to ‘does it work safely in live engineering environments under regulatory constraint?’” Those are the outlet’s words rather than the bank’s. The programme’s stated output is evidence-based guidance for large regulated organisations, which is an admission that none currently exists.

Sources: [Cloudflare](#) (20 April 2026) · [Deloitte Tech Trends 2026](#) · [GitKraken](#) (2026) · [Chris Stokel-Walker, LeadDev](#) (28 July 2026) · [Bessemer Atlas on Shopify](#) (2 April 2026) · [QA Financial via Resultsense](#) (5 May 2026)

7 OF 8

CASE STUDY

Monzo did the rebuild on purpose (2 min)

ONE BANK’S AGENT PLATFORM, AS PUBLISHED 13 AUGUST 2026

Roughly three weeks — from decision to a working MVP, built in-house rather than bought, to avoid locking the bank into one vendor’s agent.

Integrations — the agent is reachable from Slack, Linear, GitHub and engineers’ local machines.

Architecture — agents run in sandboxed containers on remote infrastructure, behind an MCP gateway.

~10% — of all merged pull requests at Monzo now come from the agent.

1,800 tasks a day — the current running volume.

Nearly all engineers — now use at least one coding agent regularly.

The bottleneck moved — from engineer capacity to reviewer availability.

Monzo published an account of Agent Chip, its internal coding-agent platform, on 13 August 2026. Two of its engineers, Fabien Deshayes and Oli Haley, wrote it. The figures are the bank's own measurement, self-reported and unaudited, and they are the closest thing in this issue to a like-for-like reference point: a UK bank, publishing its own numbers, with a named bottleneck at the end.

Monzo skipped most of stage one by building the platform layer first. The agent reaches internal knowledge sources and tools through an MCP gateway — the Model Context Protocol being a shared interface for connecting agents to internal systems, and a gateway being a single controlled entry point every such request passes through. The guardrails are architectural before they are procedural: sandboxed container isolation, an isolated proxy for external communication, per-task access configuration, environment-specific images, and secrets kept out of the agent's reach. A bank running agents against production repositories has to answer for what the agent could reach, and that list is the answer.

The agent is squarely agent-mode. The human review gate was designed in and never removed: engineers still review and merge everything it produces. That decision is what produced the finding.

“An engineer still reviews and merges, but the bottleneck moves from ‘find an engineer with capacity’ to ‘find a reviewer’. That changes who gets to improve the product.”

— Fabien Deshayes and Oli Haley, Monzo, 13 August 2026

Around 10% of Monzo's merged pull requests now come from the agent, at a running volume of 1,800 tasks a day. Both numbers describe work arriving at reviewers. Two smaller findings travel with them: non-engineers can now ship small user-experience fixes without waiting to find an available engineer, and the agent has caught errors during incident response, which puts it inside the operational loop rather than beside it.

A bank that built the verification layer alongside the platform still ended up with a review constraint. That is the wall arriving anyway, in its mildest form, and the clearest evidence here that reviewer capacity belongs in the plan from the start.

LESSONS & IMPLICATIONS

What the arc means for a team part-way along it

(2 min)

THE VERDICT

The arc is normal. A team whose output has soared while its review queue drowns has not made a mistake; it has reached stage three roughly on schedule. The wall is where the value of the whole exercise gets decided, because everything worth having afterwards is built in response to it — the test command, the instruction file, the merge gate, the cost-attribution layer. The organisations whose numbers hold up are the ones that built that layer deliberately, rather than under pressure, after the bill arrived.

Naming the stage ends most of the tool argument

Teams stuck at the wall tend to debate which agent to standardise on. The published accounts converge on scaffolding instead: a runnable test command, a short file describing how the codebase works, a stated merge gate, a single proxy carrying attribution. All four are tool-agnostic, and they are what changed the outcome in every account here.

The wall is a capacity problem before it is a quality problem

Reviewing a colleague's change means checking intent; reviewing an agent's means reconstructing it. That is why review time does not scale the way a plan assumes. The countermeasures are small and procedural: a size limit on agent pull requests, an intake standard carrying the reasoning and the test evidence, and review tiered by the risk of the code path rather than by who wrote it. Monzo found the same constraint by removing the one in front of it.

The platform work precedes the payback

Cloudflare's adoption figure came after eleven months of building. Craft Docs earned an internal proof before it issued a mandate. Sentry removed every access barrier it could name and still reports half its engineers outside daily use. The sequence is the finding, and it is the part of a rollout plan that is usually written last.

KEY TAKEAWAY

- The five stages are predictable enough to plan against, and the wall is a scheduled event on that plan.
- Reviewer capacity is the variable a rollout plan binds on, and diff size is the first lever teams reach for.
- The stated review norm and the observed one diverge quietly under volume; 79% of agent pull requests had one person doing both jobs.
- A runnable test command and a short instruction file are the cheapest scaffolding available, and every practitioner account starts there.
- Verification is a standing cost line: the 56% security pass rate has not moved in a year.
- Removing every access barrier does not produce a practice; visible peer use and workflow fit do.
- Per-user cost attribution is far cheaper to build before the token bill than after it.
- The junior-pipeline decision taken this year sets the supervision capacity available in three.

Sources: the teams, practitioners and studies cited in Stages One to Five and the case study above — among them [Addy Osmani](#), [Cloudflare](#), [Sentry](#), [Veracode](#) and [Monzo](#).

MEANWHILE ...

The Rest of the Week

While this issue was deep in the agent literature, the week kept moving.

WOULD HAVE LED

Two days of UK retail-banking degradation, and two different failure responses. On 18 August a technical fault locked Lloyds, Halifax and Bank of Scotland customers out of mobile and online banking; Downdetector logged more than 2,450 reports for Lloyds, more than 1,000 for Halifax and several hundred for Bank of Scotland, around 70% of them app-related, with problems spiking by 11:30am and service restored shortly after 1:00pm. On 19 August more than 3,000 Monzo customers reported problems after midday, and Monzo moved into its published stand-in system — a fallback mode in which core payment functions keep running while other services are unavailable. Monzo's statement: "We've moved into our back-up system Monzo Stand-in and customers can still do most things on their Monzo app including sending bank transfers and making card payments and ATM withdrawals." Neither bank has published a cause for either incident. [IBTimes UK](#) · [Evening Standard via AOL](#)

WOULD HAVE LED

Visa, Mastercard and around two dozen others form an Agentic Payments Alliance — and name loyalty as unsolved. Rain launched the alliance on 18 August with about two dozen founding members including Visa, Mastercard, Fiserv, Circle, Remitly, Shift4, Sardine, Lithic, Evertec, Episode Six and Chainalysis, to standardise agent identification and authorisation, run shared research and shape regulatory advocacy. American Banker reports that much of the infrastructure guiding agentic AI — how agents get authorised, how fraud gets caught, and how loyalty and rewards work — is still being defined. “The risk in a moment like this is not that the industry moves too slowly — it’s that innovation outpaces alignment,” said Sherri Haymond, executive vice president and global head of digital commercialization at Mastercard. [American Banker](#)

Apple patches an image-processing flaw reported by Meta’s red team. Security updates for recent iPhones, iPads, Macs and Vision Pro address an integer-overflow bug in ImageIO, the framework Apple uses to decode images, which could allow code execution when a malicious image is processed; Apple fixed it with improved input validation. [The Register](#), 18 August 2026

Meta ships Muse Code, a terminal coding agent for large codebases, in beta. Announced on 5 August, before this window opens, it fans large jobs out to parallel sub-agents working in isolated worktrees, and Meta positions it on cost against Codex and Claude Code. No pricing figures were published. [TechCrunch](#), 5 August 2026

ON THE RADAR

Coming Up

4 Aug **keyv and cacheable npm packages hijacked** — Socket reports at least ten packages in the keyv and cacheable namespaces republished with a credential-sweeping preinstall hook, collectively accounting for tens of millions of weekly downloads, after a maintainer account was compromised and stolen tokens were used to republish further packages. The leadership question is which build agents hold tokens. [Socket](#)

2 Dec 27 **EU AI Act high-risk regime slips** — Annex III stand-alone high-risk obligations covering hiring, credit scoring, education and critical infrastructure are deferred to 2 December 2027, and Annex I embedded high-risk AI to 2 August 2028. Article 50 transparency obligations were not deferred and took effect on 2 August 2026. [Gibson Dunn](#)

27 Jul **Dynatrace autonomous SRE agent and no-code Agent Builder** — announced with availability stated as August 2026; no general-availability date has been published, so this remains an announcement rather than an event. [Dynatrace](#)

RECOMMENDED READING

Sources & Further Reading

One vendor’s telemetry, attributed as such: “Claude Code sessions run on the most capable models far more often (54% are served by Opus, against 10% of chat and Cowork conversations)”, with sessions increasingly consisting of long-running agentic tasks.

Generative AI as Seniority-Biased Technological Change

Hosseini Maasoum & Lichtinger, SSRN

The working paper behind Source 4’s direction-only finding, for the authors’ own magnitudes. The SSRN page was not reachable during preparation of this issue.

Engineering Progression When AI Ate the Middle

Ben Linders, InfoQ

Alasdair Allan’s QCon London talk, connecting this year’s hiring decision to the supervision capacity available in three. Recommended rather than cited, because its figures are a secondary aggregation.

Inside AI-pilled engineering teams

Bessemer Atlas

The full account of the enablement layer summarised in Source 5, including the Ramp and Shopify practices.

ROI of AI-assisted Software Development (v.2026.1)

DORA

The model behind the finance conversation in Source 3, published with an accompanying ROI calculator.

Engineering practices that make coding agents work

Simon Willison

The transcript behind Source 2’s testing material, and the densest single hour of practice in the pack — repo readiness, session conventions and where agent-written code still needs a refinement pass.

Harness engineering for coding agent users

Birgitta Böckeler, martinowler.com

The guides-and-sensors split in full, with the argument for treating the scaffolding around the agent as a staffed, maintained artefact.

Agentic Code Review

Addy Osmani

The pull-request intake standard and risk-tiered review policy summarised in Source 2, with the tool-overlap data behind them.

VOL. II

Engineering Digest

21 August 2026 · Deep Dive · Issue #23

Next edition: 28 August 2026

ARCHIVE

#18 · Deep Dive

#19 · Weekly

#20 · Weekly

#21 · Weekly

#22 · Weekly

#23 · Deep Dive